A VISUAL KNOWLEDGE BASE · LECTURE 11

Containers Decoded

From the OCI rulebook to containerd, runc, CRI and Kubernetes: the story of what really happens when you type `docker run nginx`.

📘 OCI · 🐳 Docker internals · ⚙️ containerd · runc · 🕴️ CRI · ☸ Kubernetes

🎯 Interview Q&A



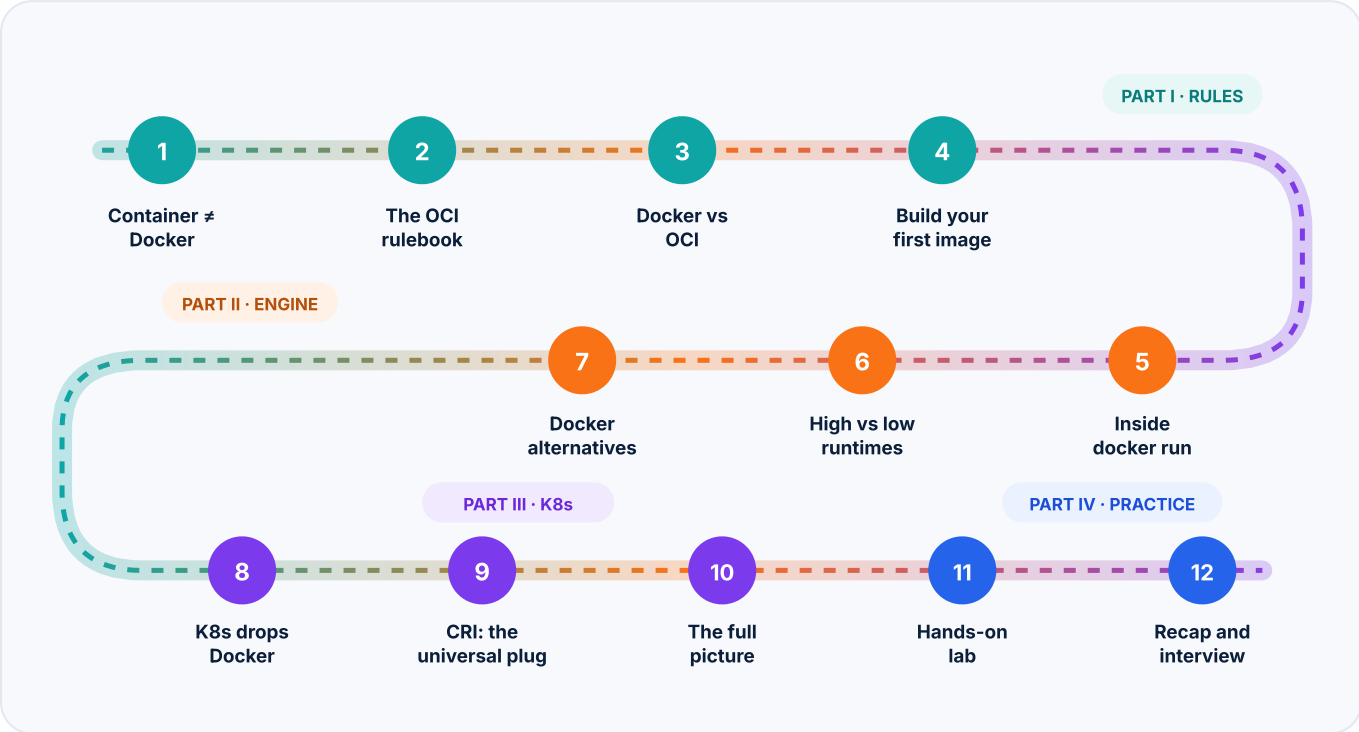
4 parts · 12 chapters · diagrams, labs and interview prep

Read it like a story, top to bottom

BEFORE WE START

Your Journey Through This Guide

> *It's 9 a.m. You type* `docker run nginx` *and press Enter. Half a second later, a web server is running. This guide is the story of that half second.*



The cast of characters

These names come up again and again. Here's who's who before the story starts:

THE FACE

🐬 Docker CLI

The command you type. It doesn't do any real work itself.

THE MANAGER

🧠 dockerd

Docker's background daemon. It handles builds, volumes and networks.

THE ENGINE

📦 containerd / CRI-O

High-level runtimes: they pull images, prepare storage and network, and manage the container lifecycle.

THE SPARK PLUG

⚙️ runc / crun

Low-level runtimes: they ask the Linux kernel to create the container process.

THE PLUG SOCKET

🔌 CRI

Kubernetes' standard API for talking to any runtime.

THE RULEBOOK

📘 OCI

Open standards for images and for how containers are run.

PART ONE

The Rules of the Game

Before anyone can ship a box around the world, everyone has to agree on what a box is.

IN THIS PART YOU'LL LEARN

- ✦ Why "container" and "Docker" are not the same thing
- ✦ What the Open Container Initiative is, and its 3 specs
- ✦ How Docker and OCI relate (car maker vs road rules)
- ✦ How to build your first OCI-compliant image



PART I · MYTH-BUSTING

01 A Container Is Not "Docker"

> *Whenever someone says "container", most people picture the Docker whale. Let's fix that first.*

Containers **existed before Docker**. Linux already had the building blocks: **namespaces, cgroups, chroot and LXC**. Docker arrived later and made containers **easy to use**. So Docker is one tool for creating containers. A container is not the same as Docker.

🪥 Plain-English analogy: "Have you done your Colgate?"

In India we often ask *"Have you done your Colgate?"* when we mean *"Have you brushed your teeth?"* Colgate is so common that it replaced the word **toothpaste**. But Colgate is just **one brand** of toothpaste. Pepsodent and others exist too. In the same way, **Docker is one brand** of container tooling.

CATEGORY: Toothpaste 🪥

- Colgate ⭐
- Pepsodent
- Sensodyne
- Close-Up

The famous brand isn't the whole category

CATEGORY: Container tools 📦

- Docker ⭐
- Podman
- containerd
- CRI-O

Docker is popular, but it's one product among many

Containers came long before Docker

1979 — **chroot**: change a process's root folder

2002 — First Linux **namespaces** (isolation)

2008 — **cgroups** in the kernel, and **LXC**

2013 — **Docker** makes it all easy 🐬

2015 — **OCI** formed: open standards

2017 — **containerd** donated to CNCF

2022 — K8s v1.24 removes **dockershim**

✅ So why did everyone start using Docker?

Because it was **very easy to use**. Pulling images, starting containers and managing them took one simple command. The technology underneath already existed. Docker gave it a friendly interface.

🎯 Interview one-liner

"A container is an **isolated, resource-limited Linux process**. Docker is one product for building and running containers."

PART I · THE RULEBOOK

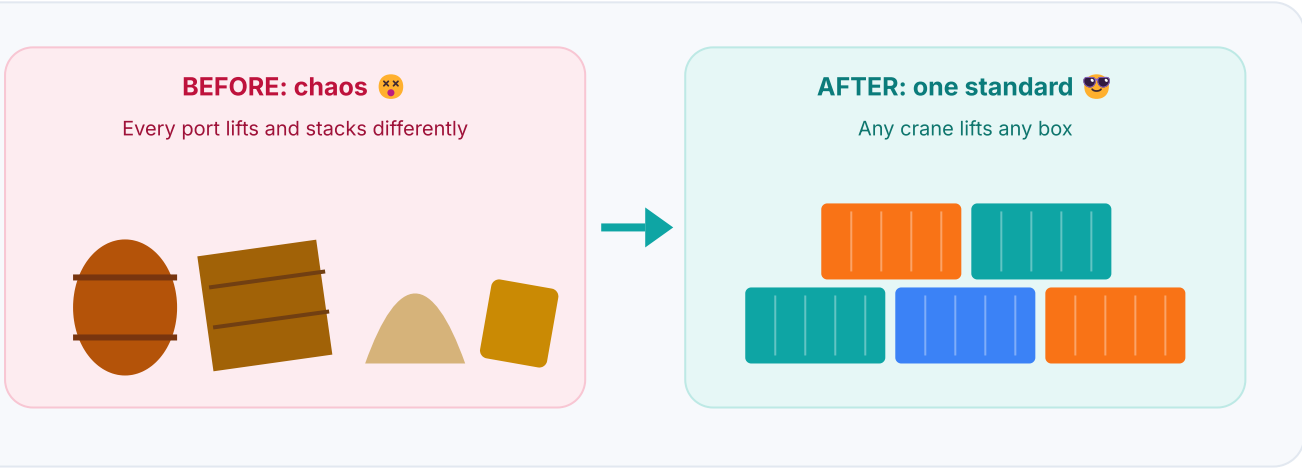
02 The Open Container Initiative (OCI)

> *If every company built its own kind of box, nothing would fit anywhere. That nearly happened.*

The **Open Container Initiative (OCI)** is an industry body under the **Linux Foundation** that sets the **universal standards** for container software. In short, it's the rulebook every container tool agrees to follow.

⚓ **Plain-English analogy: shipping containers**

In the old days, goods travelled in random crates, barrels and sacks, and every port handled them differently. Then the shipping industry agreed on **one standard box**: exact dimensions, the same corner locks and a weight limit. Now **any crane in any port can lift any container**, whether it holds electronics, clothes or cars.



How this applies to software

A software **container** is a digital box. Inside it, a developer packs the **code** along with all the **tools, libraries and settings** it needs. Before OCI, there was a real risk that **Docker, CoreOS, Google** and others would each create their own formats that didn't work together. So in **2015**, Docker, CoreOS and others formed the OCI. Docker donated its **image format** and **runc** (the reference runtime) to it.



1 · IMAGE SPEC

📦 The Box

The **format** of an image: its layers, manifest and config.



2 · RUNTIME SPEC

🏗️ The Crane

How a runtime must run a container from an unpacked **bundle** (`config.json` plus a root filesystem).



3 · DISTRIBUTION SPEC

🚢 The Ship

How **registries** (Docker Hub, ECR, ACR) store and serve images.



Build once, run anywhere: the same box fits every ship and every crane.

Key takeaway

Once an image follows the OCI standard, it runs on **any compliant runtime**: containerd, CRI-O, Podman and others. **runc** is the reference OCI runtime, and **crun** is OCI-compatible too.

PART I · WHO'S WHO

03 Docker vs OCI: Product vs Rules

Docker started the container boom. So why does it follow someone else's rules?

🚗 **Plain-English analogy: car makers and road rules**

Docker is like a **car manufacturer** (think Toyota). **OCI** is the **regulatory body** that sets universal road standards, such as which side of the road to drive on or the shape of a number plate.

ACT 1 · THE ORIGIN

Docker leads

Docker made the modern container popular. Because it owned the technology, the industry worried that Docker would control all the rules.

ACT 2 · THE SOLUTION

Docker shares

To prevent a monopoly, Docker **donated its core technology** (image format and runc) to the community. That created the OCI.

ACT 3 · TODAY

Product vs rules

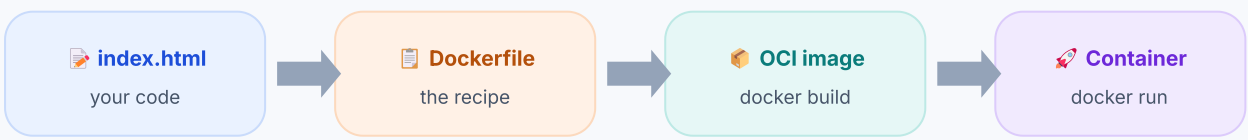
Docker is a product you install. **OCI** is the set of rules Docker and its competitors all follow.

	🐳 Docker	📘 OCI
What it is	A tool or product	A standards body (Linux Foundation) and its specifications
You can...	Install it and run commands	Read it. Tools implement it.
Analogy	Toyota (car maker)	Road and number-plate regulations
Example	`docker build`, `docker run`	Image Spec, Runtime Spec, Distribution Spec

PART I · HANDS-ON

04 Build Your First OCI-Compliant Image

This is **easier than it sounds**. Your everyday tools handle the OCI rules in the background.



1 Write the software

index.html

```
<h1>Hello, World! This is an OCI container!</h1>
```

2 Write the recipe (Dockerfile)

Dockerfile

```
# OCI-compliant base image
FROM nginx:alpine
# copy the website in
COPY index.html /usr/share/nginx/html/
# open port 80
EXPOSE 80
```

3 Build the box

terminal

```
$ docker build -t my-first-oci .
```

Swap `docker` for `podman` or `buildah` and it works the same way.

4 Run it

terminal

```
$ docker run -d -p 8080:80 my-first-oci
```

Open `http://localhost:8080` 🎉 The same image runs on Kubernetes, AWS or GCP unchanged.

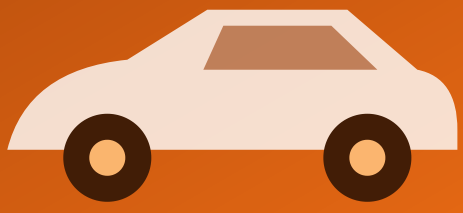
PART TWO

Under the Bonnet

Docker looks like a single thing, like a car. Open the bonnet and you'll find an engine, a gearbox and a spark plug working together.

IN THIS PART YOU'LL LEARN

- ✦ What really happens on `docker run nginx`
- ✦ The 4 components: CLI, dockerd, containerd, runc
- ✦ What a container runtime is: high-level vs low-level
- ✦ Docker alternatives: Podman, Finch, Buildah, Skopeo



05

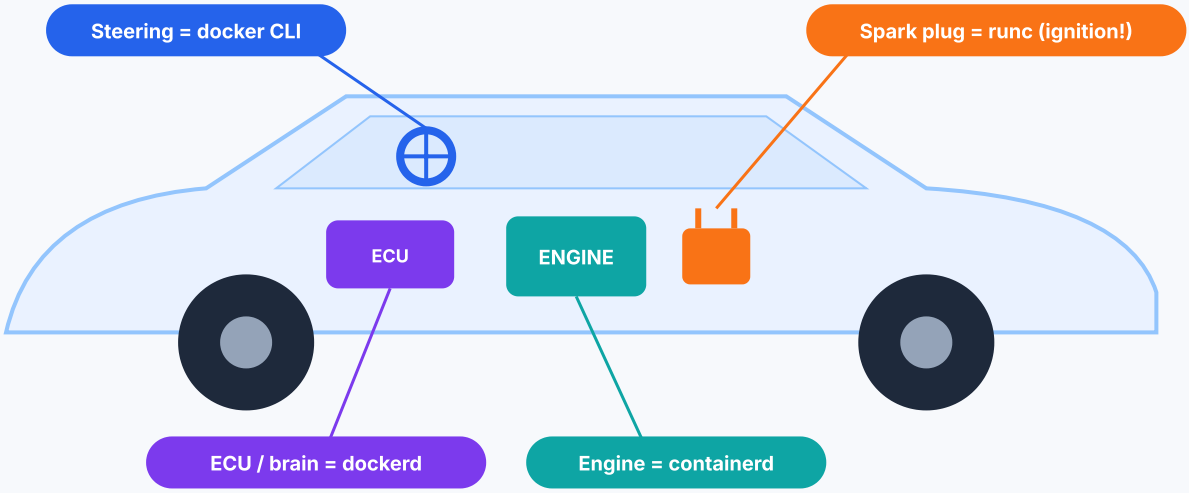
PART II · THE BIG REVEAL

What Really Happens on `docker run nginx`

You type one command. docker ps shows nginx running. Simple? Behind the curtain, four different programs just passed the job along like a relay team.

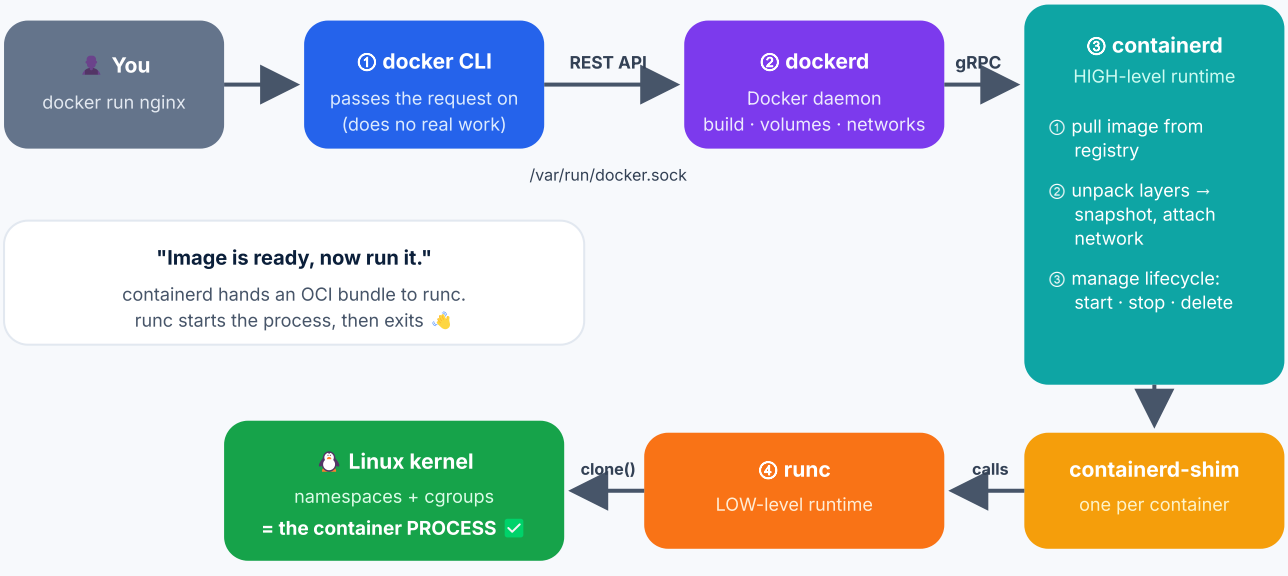
Plain-English analogy: Docker is a car

A car is a combination of parts: wheels, a steering wheel, an engine and more. When they all work together we just call it "a car". **Docker is the same.** Several components are hidden from us, and we only see "Docker".



You hold the steering wheel. The other parts do the work out of sight.

The relay race: who does what



COMPONENT 1

🖥️ Docker CLI

The `docker …` command we type. It **does no real work**. It passes the request to the daemon.

That's why a stopped daemon gives this error:

```
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?
```

COMPONENT 2

🧠 Docker daemon (dockerd)

A **daemon** is a long-running background process. The word comes from Greek mythology, where a *daimon* was a helpful spirit working quietly in the background. 👻

dockerd receives the CLI request and passes it to the runtime. It also handles **Docker-only features**: `docker build`, volumes, networks, Compose and Swarm.

COMPONENT 3

📦 containerd (high-level)

Always running alongside dockerd. **Created by Docker**, donated to the **CNCF in 2017** (graduated in 2019), and now used well beyond Docker.

It **pulls images**, **sets up storage and network**, and **manages the lifecycle**. Then it tells runc: "Image is ready, please run it."

COMPONENT 4

⚙️ runc (low-level)

The program that **actually runs the container**. It uses Linux **namespaces** (PID, network, mount, UTS, IPC, user) for isolation and **cgroups** for CPU and memory limits to create **one process**.

That process **is** the container.

💡 The "aha" moment

Every time you run a container, a **Linux process** is created. A container is just an **isolated, resource-limited Linux process**. There's no magic.

🎯 Interview detail: the containerd-shim

containerd starts a small **containerd-shim** process for each container. The shim calls runc, **runc creates the container and exits**, and the **shim stays** as the container's parent. It holds STDIO and reports the exit code. That's why you can **restart containerd or dockerd without killing running containers**. Check it with `ps -ef | grep containerd-shim`.

PART II · DEFINITIONS

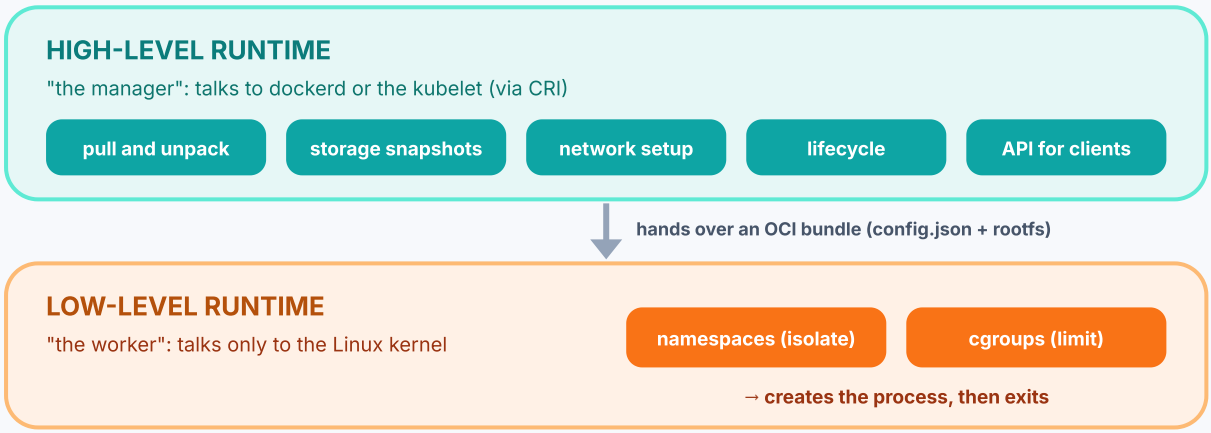
06 What Is a Container Runtime?

> *We keep saying "runtime". Let's pin down what it means, and why there are two levels of it.*

A **container runtime** is the software that **creates and runs containers**.

Plain-English analogy: the Python runtime

To run a Python app you need the **Python runtime** (the interpreter). To run a container you need a **container runtime**. No runtime, no container.



	High-level runtime	Low-level runtime
Job	Image pull and unpack, storage snapshots, network setup, container lifecycle, API for clients	Takes an unpacked bundle and config, and creates the process using namespaces and cgroups
Talks to	Docker daemon / kubelet (via CRI)	The Linux kernel
Examples	**containerd** (Docker, most K8s clusters) **CRI-O** (Red Hat / OpenShift, pronounced "cry-oh")	**runc** (default, written in Go) **crun** (Red Hat, written in C, faster and lighter) Sandboxed: **gVisor** (runsc), **Kata Containers**

What about Firecracker?

Firecracker (by AWS) powers **Lambda and Fargate**. Strictly speaking it's a **microVM monitor**, not a runc replacement. Each function or task gets a tiny, fast-booting VM for stronger isolation. It plugs into containerd through `firecracker-containerd`.

The summary slide, in one view

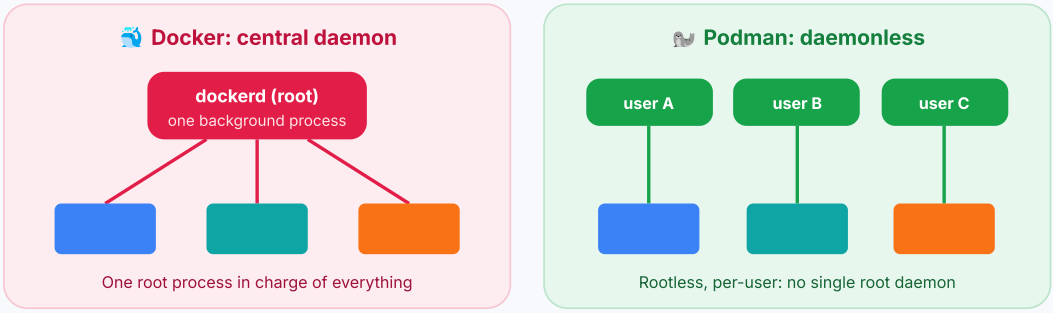
docker CLI	What you type
docker daemon	Docker's background brain (dockerd)
High-level	**containerd**, **CRI-O**
Low-level	**runc**, **crun**
CRI	Container Runtime Interface, defined by **Kubernetes**
OCI	Image specification and runtime specification

PART II · BEYOND DOCKER

07 Docker Alternatives That Speak OCI

Because everyone follows the same rulebook, you're free to choose a different tool.

Why "daemonless" matters



BY RED HAT

Podman (Pod Manager)

What it is: a drop-in, Docker-compatible replacement that uses crun or runc underneath.

Why use it: it's **daemonless and rootless**. Containers run under the user's own account rather than through one root daemon, so it's more secure.

Plus side: `alias docker=podman` and most of your usual commands work.

BY AWS

Finch

What it is: an open-source CLI for **macOS and Windows** developers.

Why use it: it bundles curated OCI tools (`nerdctl`, `lima`) into one clean install.

Plus side: it's tuned for building locally and deploying straight to **AWS**.

Tool	Best for	Daemon?	Build	Run
Docker	All-in-one developer experience	Yes	✓	✓
Podman	Secure, rootless Docker replacement	No	✓	✓
Finch	Mac/Windows developers targeting AWS	Bundled VM	✓	✓
nerdctl	Docker-like commands directly on containerd	Uses containerd	✓	✓
Buildah	Does one job: **building OCI images**. Ideal in CI without a full Docker install	No	✓	–
Skopeo	**Inspects, copies and signs** images between registries without downloading the whole image	No	–	–

PART THREE

Enter Kubernetes

Kubernetes looked at Docker's car and said: "I don't need the steering wheel or the dashboard. Just give me the engine."

IN THIS PART YOU'LL LEARN

- ✦ Why Kubernetes dropped the Docker layer (dockershim)
- ✦ How to work with containers on a node without Docker
- ✦ CRI: the universal plug socket for runtimes
- ✦ CRI vs OCI, and the full picture side by side



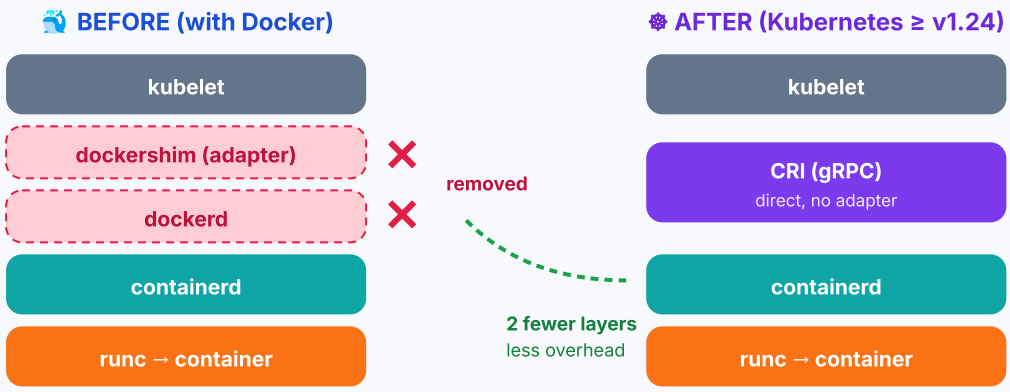
PART III · THE BREAKUP

08 Why Kubernetes Dropped the Docker Layer

> *Kubernetes asked a fair question: "Why should I carry a CLI and a daemon I never use?"*

In Docker the chain is **CLI → dockerd → containerd → runc**. Kubernetes' view: *"The Docker CLI and daemon are useless to me. They **eat CPU and memory** for nothing. I only need your runtime, **containerd**."*

On a Kubernetes node, the **kubelet is the "user"**. It already does what the CLI and daemon did: it receives Pod specs from the API server and decides what to run.



History for interviews: the "Kubernetes is dropping Docker" story

Docker **never implemented CRI**, so Kubernetes had to maintain an adapter called **dockershim** inside the kubelet. It was **deprecated in v1.20** and **removed in v1.24 (2022)**. Images built with `docker build` **still work everywhere** because they're OCI images. Only Docker as the *node runtime* was removed. Mirantis maintains `cri-dockerd` if you really need it.

So how do I see containers on a node?

The Docker CLI and daemon aren't on the node, so `docker ps` doesn't exist there. Use these instead:

Tool	From	Use it for
⭐ **crictl**	Kubernetes (cri-tools)	Talks CRI, so it works with **containerd or CRI-O. The one to use for K8s troubleshooting**: `crictl ps`, `crictl pods`, `crictl logs`
ctr	Ships with containerd	Low-level debug client and not user-friendly. K8s containers live in the `k8s.io` namespace: `ctr -n k8s.io containers ls`
nerdctl	containerd sub-project	Docker-compatible CLI for containerd: `nerdctl run -d nginx`
podman	Red Hat	Daemonless, Docker-compatible CLI (uses crun or runc)

09

PART III · THE INTERFACE

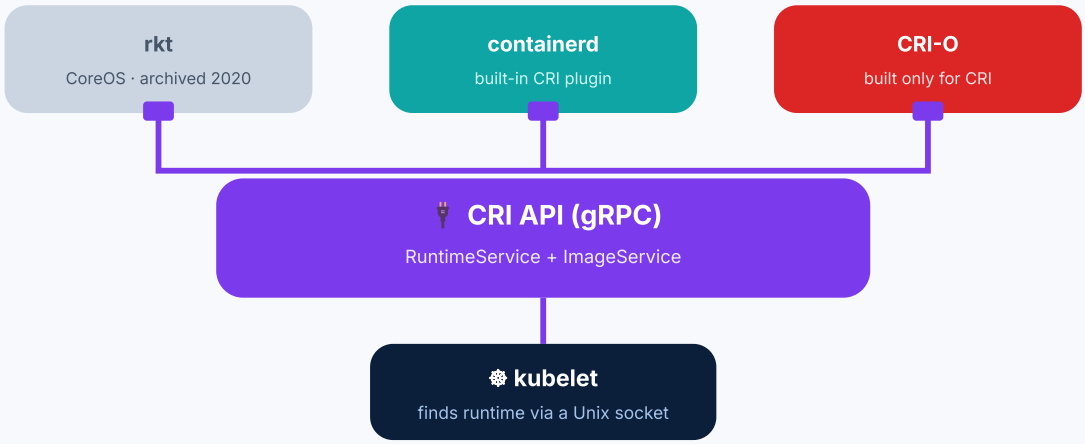
CRI: The Universal Plug Socket

> *Several runtime makers knocked on Kubernetes' door: "Why always containerd? How do we connect?"*

CRI = Container Runtime Interface: a set of APIs **created by Kubernetes** for talking to container runtimes. The **problem** it solved: besides containerd there were other runtimes, such as **rkt** ("Rocket", by CoreOS) and later **CRI-O**. Hard-coding each one into the kubelet wasn't sustainable.

Plain-English analogy: one wall socket for every appliance

Your house doesn't rewire itself for each appliance. It has a **standard socket**, and any appliance with the right plug just works. CRI is that socket. Its message to every runtime is: *"Follow my guidelines, and only then can you talk to Kubernetes."*



GRPC SERVICE 1

RuntimeService

Run and stop **Pod sandboxes** and containers, plus **exec** and **logs**.

GRPC SERVICE 2

ImageService

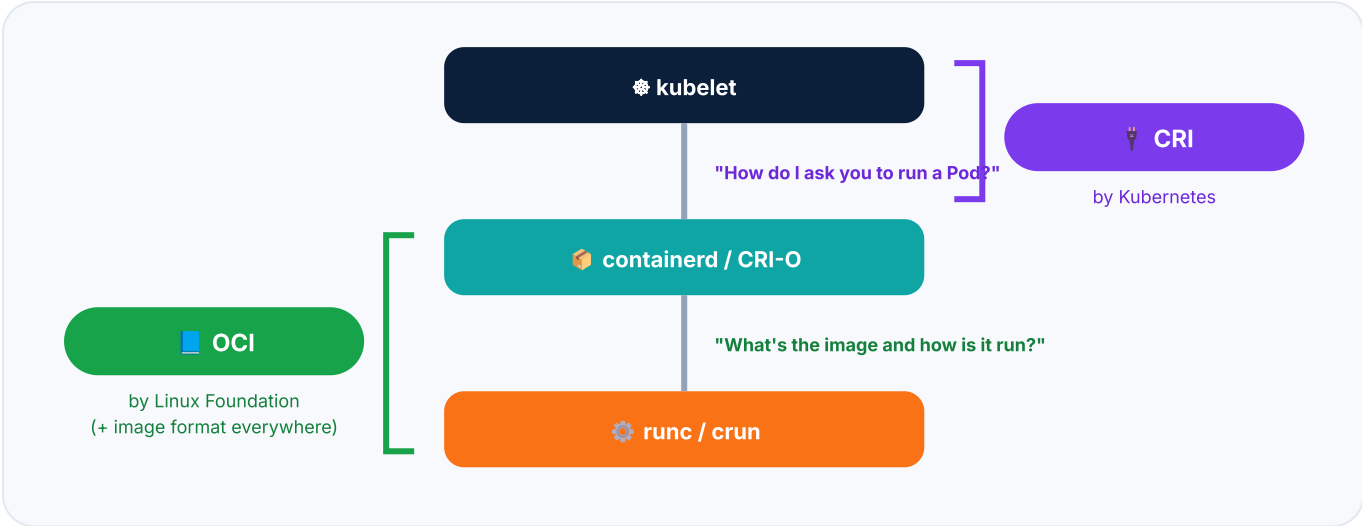
Pull, list and remove images.

how the kubelet finds its runtime

```
# containerd
--container-runtime-endpoint=unix:///run/containerd/containerd.sock
# CRI-O
--container-runtime-endpoint=unix:///var/run/crio/crio.sock
```

CRI vs OCI: don't mix them up

This is the most common mix-up in interviews. The diagram shows **exactly where each one sits**:

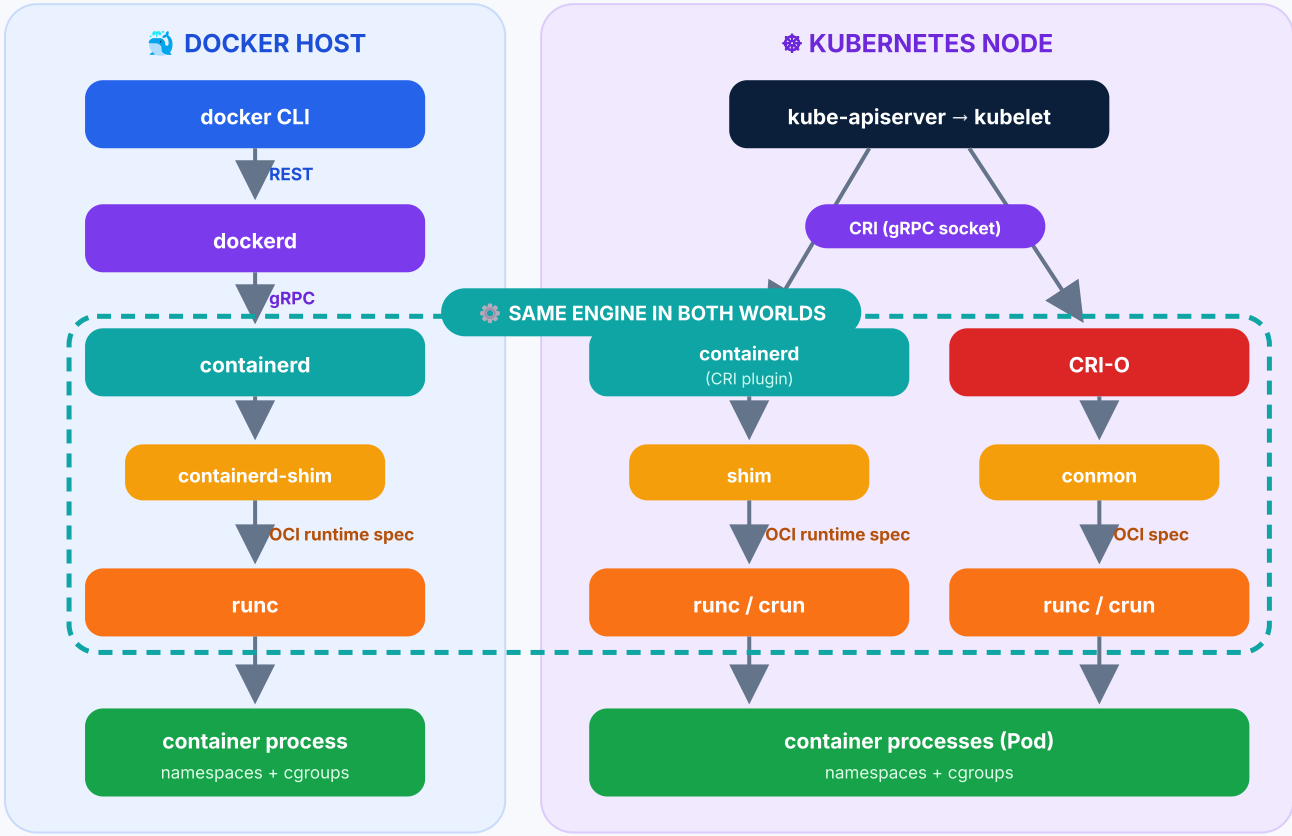


	CRI	OCI
Defined by	Kubernetes	Open Container Initiative (Linux Foundation, started by Docker and others)
Standardises	The **API** between the kubelet and a high-level runtime	The **image format** and how a low-level runtime runs a container
Sits between	kubelet ↔ containerd / CRI-O	containerd / CRI-O ↔ runc / crun (and builders ↔ registries ↔ runtimes)
One-liner	"How Kubernetes talks to a runtime"	"What an image is and how a container is run"

PART III · EVERYTHING TOGETHER

10 The Full Picture: Docker Host vs Kubernetes Node

Lay the two worlds side by side and something becomes clear: the engine is the same.



Key takeaway: the whole story in one sentence

containerd + runc are the same engine in both worlds. Kubernetes simply **removed the Docker "car body"** (CLI and daemon) and **drives the engine directly through CRI**.

The Kubernetes chain, step by step

1 **Request**

The API server sends a Pod spec. The **kubelet** calls the runtime over **CRI**.

2 **Manager**

containerd or CRI-O pulls the OCI image, unpacks the layers and sets up the network.

3 **Executor**

runc asks the kernel for namespaces and cgroups, starts the process and **exits**.

Why this matters

Kubernetes **doesn't care** whether your image was built with Docker, Podman or Finch. If it meets the OCI spec, **containerd can pull it and runc can start it.**

PART FOUR

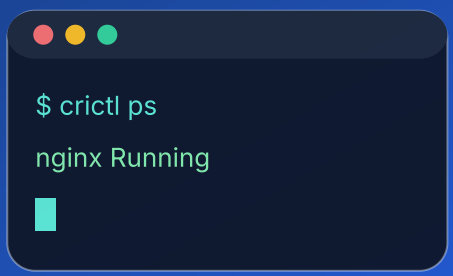
IV

Hands On and Exam Ready

Theory makes sense on paper. Commands make it stick.

IN THIS PART YOU'LL FIND

- ✦ A lab sheet for containerd on a Kubernetes node
- ✦ A Docker → crictl command translator
- ✦ A one-page cheat sheet
- ✦ 10 interview questions with crisp answers



PART IV · LAB

11 Hands-on Reference: containerd on a K8s Node

SSH into a worker node. There's no Docker here. Let's look around anyway.

1 · is the runtime running?

```
$ systemctl status containerd
$ kubectl get nodes -o wide
# see the CONTAINER-RUNTIME column
```

2 · crictl: point it at the socket once

```
$ cat <<'EOF' > /etc/crictl.yaml
runtime-endpoint:
unix:///run/containerd/containerd.sock
image-endpoint:
unix:///run/containerd/containerd.sock
EOF
```

🐳 You know this…	☸ …on a K8s node use this	What it shows
`docker ps`	`crictl ps`	Running containers
(no equivalent)	`crictl pods`	Pod sandboxes on this node
`docker images`	`crictl images`	Images on this node
`docker pull nginx`	`crictl pull nginx`	Pull through CRI
`docker logs <id>`	`crictl logs <id>`	Container logs
`docker exec -it <id> sh`	`crictl exec -it <id> sh`	Shell inside the container
`docker inspect <id>`	`crictl inspect <id>`	Full details

3 · ctr: containerd's low-level client

```
$ ctr namespaces ls
  # default, k8s.io, moby (Docker's)
$ ctr -n k8s.io images ls
$ ctr -n k8s.io containers ls
```

4 · nerdctl: Docker-like on containerd

```
$ nerdctl run -d --name web \
    -p 8080:80 nginx
$ nerdctl ps
```

5 · prove a container is "just a process"

```
$ ps -ef | grep -E 'containerd-shim|nginx'
```

6 · containerd config for kubeadm clusters

```
$ containerd config default > /etc/containerd/config.toml
# set SystemdCgroup = true under:
# [plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc.options]
$ systemctl restart containerd
```

! Common pitfall: cgroup driver mismatch

containerd and the kubelet must use the **same cgroup driver (systemd)**. A mismatch makes nodes unstable and Pods go into **CrashLoopBackOff**. Set `SystemdCgroup = true`.

12

PART IV · REVISE IN 5 MINUTES

Recap and Interview Q&A

The one-page cheat sheet

Container	An **isolated, resource-limited Linux process** (namespaces + cgroups). Not the same as Docker 🪒.
OCI	The **rulebook** (Linux Foundation, 2015): **Image** 📦, **Runtime** 🏗️ and **Distribution** 🚢 specs.
Docker chain	CLI →(REST, docker.sock)→ **dockerd** →(gRPC)→ **containerd** → shim → **runc** → process.
High-level	**containerd**, **CRI-O**: images, storage, network, lifecycle.
Low-level	**runc**, **crun** (gVisor, Kata for sandboxing): create the process, then exit.
CRI	Kubernetes' **gRPC API** (RuntimeService + ImageService) between the kubelet and the runtime.
K8s chain	kubelet →(CRI)→ **containerd / CRI-O** → shim / conmon → **runc / crun** → Pod processes.
dockershim	Deprecated in v1.20, **removed in v1.24 (2022)**. Docker-built images still run (OCI).
Node tools	**crictl** (K8s, use this), **ctr** (containerd debug), **nerdctl**, **podman**.

Interview questions with crisp answers

1. What happens on docker run nginx?

CLI → dockerd (REST over docker.sock) → containerd (pulls image, prepares snapshot and network) → shim → runc (namespaces + cgroups) → a Linux process, which is the container. runc exits and the shim stays.

2. Container vs Docker?

A container is an isolated Linux process. Docker is one product for building and running containers (the Colgate analogy).

3. What is OCI, in one line?

Open standards for how images are packaged, distributed and run, so any compliant tool works with any compliant image.

4. High-level vs low-level runtime?

containerd and CRI-O manage images and the lifecycle. runc and crun just create the process according to the OCI runtime spec.

5. Why did Kubernetes remove Docker?

Docker didn't implement CRI, maintaining dockershim was a burden, and the CLI and daemon add overhead. It was removed in v1.24. Docker-built images still run.

6. CRI vs OCI?

CRI is the API between the kubelet and the runtime, defined by Kubernetes. OCI covers the image and runtime format standards, under the Linux Foundation.

7. Debug containers on a node without Docker?

Use crictl (ps, logs, exec, inspect), plus journalctl -u containerd and journalctl -u kubelet.

8. Why can you restart containerd without killing containers?

Each container's parent is its containerd-shim, not containerd itself. runc has already exited.

9. Why choose Podman over Docker?

It's daemonless and rootless, so there's no single root daemon, and its commands are Docker-compatible.

10. Building images in CI without Docker?

Use Buildah (or Kaniko) to build, and Skopeo to copy, inspect or sign images between registries such as ECR and ACR.

🔭 Still to come (next chapters of your learning)

- Installing containerd for a kubeadm cluster step by step (modules, sysctl, `SystemdCgroup`)
- Pod sandbox and the **pause container**: why every Pod has one extra container
- **RuntimeClass**: running some Pods with gVisor or Kata for stronger isolation

📦 Build once · 🚢 Ship anywhere · ☸ Run everywhere